Lecture 1: Motivation and Recap of Turing Machines

Lecturer: Ari

1. Course structure

We begin the complexity theory part of our course. The material is adapted from the undergraduate complexity theory course at University Of Oxford, taught by Rahul Santhanam. We will only cover the following topics.

1. Motivation and recap of Turing machines [2 lectures]
2. Time and space [2 lectures]
3. Non determinism [4 lectures]

Each lecture consists of 60 minutes of whiteboard teaching.

1.1. Notation

Below is some general notation we will use throughout these notes. We make use of some general notation and shorthand throughout this course, which we explicitly define here for reference. We write $[n]$ to denote $\{1, 2, \ldots, n\}$, and $[a, b]$ to denote the interval $\{a, a+1, \ldots, b-1, b\}$

2. Motivation

Complexity theory is about answering questions such as:

1. What makes one algorithm for solving some problem better than another?
2. What is the best possible algorithm for solving some problem?
3. How does the difficulty of solving one problem compare to the difficulty of solving another, i.e. can we quantify how much harder one problem is than another?

Before one can answer such questions one must clearly define what an algorithm is, and what makes an algorithm better than another. The definition of an algorithm gives us our *model of computation* and betterness is given by some measure of *complexity*. Consider the problem of sorting a list of n items. Intuitively, to sort things, we need to compare one item with another to decide which comes first. Thus, it might seem natural to define an algorithm as a deterministic black-box that can compare items.

> **Definition 1.1.** A comparison-based sorting algorithm takes as input an list $\vec{a} = (a_1, a_2, \ldots, a_n)$ of $n \in \mathbb{N}$ items from a set with total order $<$, and can only gain information about the items by comparing pairs of them. Each comparison ("is $a_i < a_j$?") returns YES or NO and counts as one time step. The algorithm may also reorder items for free based on the results of comparisons made. In the end, the algorithm must output a permutation of the input in which all items are in sorted order.

Such an algorithm definition is not general-purpose. Or at least it does not seem general purpose, it is only good for sorting things. But it is still a precisely defined model, and we are able to analyse it.

Figure 1 gives a mental model of how to think about such an algorithm. The final output is a permutation $\pi : [n] \to [n]$ that tells us how to re-arrange the indices to sort the input. A

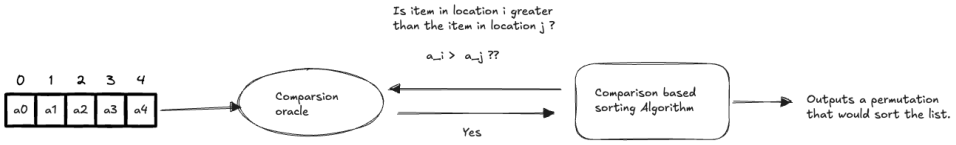


Figure 1: The comparison based algorithm cannot see the contents of the array. It decides a sequence of questions it must ask. For example, its first query might be *is the item at position 4 smaller than the item in position 9.* Based on whether the answer was YES or NO, it might then ask *is the item in position 6 smaller than the item in position 9.* It goes on asking questions and receiving answers till it outputs a permutation function. The complexity of such an algorithm is the number of questions it asks.

permutation π moves the element a_i from position i in the input to position $\pi(i)$ in the final output. Note that the logic for the next question is given by some deterministic function that uses as input the previous questions and previous answers. We list some well-known comparison-based sorting algorithms and upper bounds on the number of comparisons they need:

- Merge sort: $O(n \log n)$
- Heap sort: $O(n \log n)$
- Quick sort: amortised $O(n \log n)$, but $O(n^2)$ in the worst case
- Bubble sort, selection sort: $O(n^2)$

But how do we know that $O(n \log n)$ is the best we can do?

Theorem 1.2. *Any* deterministic *comparison-based sorting algorithm must perform* $\Omega(n \log n)$ *comparisons to sort* n *elements in the* worst case.

Proof. We want to show that, whatever deterministic comparison-based sorting algorithm we choose, some input forces it to make at least $\log_2(n!)$ comparisons. Fix any such algorithm. Our task is to find some input $\vec{a} = (a_1, \ldots, a_n)$ that forces $\log_2(n!)$ comparisons.

As we are choosing the input, we can assume that the array $\vec{a} = (a_1, \ldots, a_n)$ contains distinct elements. Let S_n denote the set of all permutations $\pi : [n] \to [n]$. A key feature of assuming distinct elements is that **every** input list of n items is sorted by exactly one permutation $\pi \in S_n$. In other words, the map from inputs to permutations that sort them is a bijective map. For a comparison query "Is $a_i < a_j$?", we say that π is **consistent with the answer** if:

- The answer is **YES** and $\pi(i) < \pi(j)$; or
- The answer is **NO** and $\pi(i) \geq \pi(j)$.

We say π is consistent with all q comparison results if the above condition holds for each of the q different comparison queries and its answer. Set $H_0 := S_n$, the set of all possible permutations. Define H_q to be the set of permutations consistent with the results of all $q \geq 0$ comparisons so far.[1] The next comparison query $a_i < a_j$ chosen by the sorting algorithm partitions H_q into two sets: those permutations $\pi \in S_n$ for which $\pi(i) < \pi(j)$, and the remaining permutations. Define H_{q+1} to **always** be the larger of the two sets, breaking ties arbitrarily. If we keep the set where $\pi(i) < \pi(j)$, answer YES to the algorithm's query; otherwise, answer NO. By a simple counting

[1]Note that the first query q_1 is fixed. We choose $r_1 \in \{\text{YES}, \text{NO}\}$ based on which answer shrinks H_0 the least. Then q_2 is selected as a deterministic function of q_1 and r_1. This process goes on k times. q_k is selected as a deterministic function of $(q_1, r_1, \ldots, q_{k-1}, r_{k-1})$.

argument we have $|H_q|/2 \leq |H_{q+1}| \leq |H_q|$. Now suppose the algorithm makes k queries and then terminates with output permutation σ.

As the map from inputs to permutations is a bijection, for every possible input, there is a unique σ it needs to output. As per Definition 1.1, the only information the algorithm has to output $\sigma \in S_n$ is the sequence of query-answer pairs it has received in the game above. Recall, for every query we adversarially picked the answer that kept at least half of the permutations consistent with the set of questions and answers so far. Let's call this adversarial sequence we generated $(q_1, r_1), \ldots, (q_k, r_k)$. Now we claim that $|H_k| = 1$.

> Assume not. Since H_k is nonempty by construction, it contains at least two distinct permutations. We can therefore choose $\pi \in H_k$ with $\pi \neq \sigma$. Observe that π is consistent with $(q_1, r_1), \ldots, (q_k, r_k)$. Choose an input $\vec{x}$ by setting $x_i = \pi(i)$ for each $i \in [n]$ i.e π sorts $\vec{x}$. Running the algorithm on this input therefore produces the same sequence of queries and answers $(q_1, r_1), \ldots, (q_k, r_k)$ (as the algorithm deterministic and π and σ are consistent). So the algorithm outputs σ as the answer after k queries based on our assumption. But the only permutation that sorts this input $\vec{x}$ is π. Since $\sigma \neq \pi$, the algorithm fails to sort this input, contradicting our assumption that this algorithm can sort any input. Therefore $|H_k| = 1$.

Let π_* be the unique permutation in H_k. That is, we choose the input by setting $a_i = \pi_*(i)$ for each $i \in [n]$. Since each query shrinks the size of H_q by at most a factor of 2, we start with $|H_0| = n!$ and reach $|H_k| = 1$ after k queries, so we have

$$1 \geq \frac{n!}{2^k} \implies k \geq \log_2(n!).$$

Writing out both steps, for $n \geq 4$,

$$\begin{aligned}
\log_2(n!) &= \log_2 1 + \log_2 2 + \cdots + \log_2 n \\
&\geq \underbrace{0 + \cdots + 0}_{\lfloor n/2 \rfloor \text{ terms}} + \log_2(\lfloor n/2 \rfloor + 1) + \cdots + \log_2 n \\
&\geq \log_2(n/2) + \cdots + \log_2(n/2) \\
&= \lceil n/2 \rceil \log_2(n/2) \\
&\geq \frac{n}{2} \log_2(n/2) \\
&\geq \frac{n}{4} \log_2 n = \Omega(n \log n)
\end{aligned}$$

□

Now consider the counting sort algorithm[2], which uses no comparisons at all, and instead leverages *space* as the measure of complexity.

> Given an input $[a_1, \ldots, a_n]$, the algorithm scans the list once and computes
>
> $$a^* = \max_{i \in [n]} a_i.$$
>
> It then creates a^* counters, initialised to zero, and scans the input again; each time it meets an element it increments the counter of the bin for that element. Finally, it outputs the sorted list by emptying the bins in order.

[2] https://en.wikipedia.org/wiki/Counting_sort

So how do we compare the complexity of pigeonhole sort to that of, say, merge sort? We cannot do so using "number of comparisons" as the complexity measure. We need a more general model of computation, and the one we use in this course is the multi-tape Turing machine.

3. A general model of computation

Input representation. Different problems have different inputs, like integers, matrices or graphs. As we want to compare a diverse set of problems, we represent the input to a problem as a string over some alphabet Σ. When $\Sigma = \{0, 1\}$, input x is a bit string in $x \in \{0, 1\}^*$. Any integer, graph, matrix or function can be encoded as a bit string, so this representation captures all the inputs we might care about.

Multi-tape Turing machines. Informally, a k-tape Turing machine (TM) realises a scratch pad together with a finite set of rules for operating on it.

Scratch pad. The scratch pad consists of k tapes, each a line of cells that is infinite in one direction. Each cell holds a symbol from a finite set Γ, the *tape alphabet*. Each tape i has a head p_i that can read or write one cell at a time. Time is divided into discrete steps, in each of which one unit of computation takes place. The first tape is the input tape and is read-only. The remaining $k - 1$ tapes are read-write work tapes, and the last of them is often called the output tape.

Rules. The machine has a finite set of states Q, and at any time step it is in a single state $q \in Q$. The state together with the contents of the tapes under the heads determines the next action. A single action consists of:

1. reading the symbols in the cells under the k heads;
2. replacing the symbol on each read-write tape with a new one (possibly the old symbol again);
3. changing state from q to some $q' \in Q$ (possibly $q' = q$);
4. moving each head one cell to the left (`L`), right (`R`) or stay (`S`).

Definition 1.3 (Turing machine). A TM is a tuple

$$(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}}),$$

where

- Q is the set of states the machine can be in;
- Σ is the set of symbols that can appear on the input tape;
- Γ is the set of symbols that can appear on the work tapes. It contains a designated blank symbol $\square$ and includes Σ;
- $\delta : Q \times \Gamma^k \to Q \times \Gamma^{k-1} \times \{\texttt{L}, \texttt{R}, \texttt{S}\}^k$ is the transition function, describing the next step from the current state and tape contents;
- $q_0 \in Q$ is the initial state;
- $q_{\text{accept}} \in Q$ is a state in which the TM halts and accepts;
- $q_{\text{reject}} \in Q$ is a state in which the TM halts and rejects.

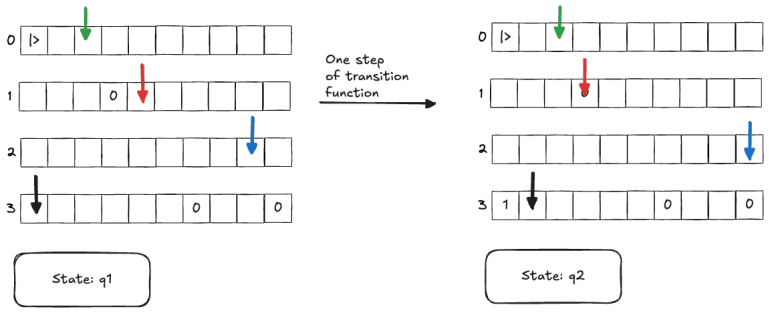


Figure 2: A k-tape Turing machine with alphabet $\{0, 1\}$ doing one state transition, or one time step taking the TM from configuration C to C'. The pointer heads $(p_i)_{i\in[k]}$ are shown with coloured arrows, and they can move left, right or stay in the current location.

Definition 1.4 (Configuration). The configuration of a TM at a given time step is a tuple $C = (q, (w_1, \ldots, w_k), (p_1, \ldots, p_k))$, where $q \in Q$ is the current state, w_i is the contents of tape i, and $p_i \in \mathbb{N}$ is the cell under the head of tape i.

For configurations C and C' we write $C \mapsto C'$ if C' is obtained from C by applying δ once, and $C \overset{*}{\mapsto} C'$ if it is obtained by applying δ finitely many times.

Definition 1.5 (Acceptance). Let $C_0(x)$ be the initial configuration of a TM M on input x. The machine M accepts x if and only if there exists a configuration C_{accept} with state q_{accept} such that

$$C_0(x) \overset{*}{\mapsto} C_{\text{accept}}.$$

Definition 1.6 (Language of a TM). Given a TM M, its language $L(M)$ is the set of inputs $x \in \Sigma^*$ that M accepts:

$$L(M) = \{x \in \Sigma^* : M \text{ accepts } x\}.$$

Remark. It looks as if a TM can only check whether an input x is in a language, i.e. answer YES or NO. In the sorting example above, however, we want the sorted array as output. In that case one treats the last tape as an output tape, and when the TM reaches an accepting state, the contents of the output tape are the output of the computation.